Proyecto Final: Tercer Entregable

Dashboard Predictivo, Modelo Original e Informe Reproducible

Pregrado en Ciencia de Datos

Guía Técnica, Entregables y Rúbrica de Evaluación

---

1. Introducción

El tercer y último entregable cierra el proyecto de Machine Learning. Mientras que el primer entregable se centró en el ETL y el análisis exploratorio, y el segundo en la construcción, optimización y comparación estadística de 140 modelos, este tercer entregable exige **llevar los mejores resultados a producción** y, además, **aportar un modelo propio** que demuestre capacidad de investigación. El entregable consta de cuatro componentes obligatorios:

1. **Correcciones del Entregable 2:** Todas las observaciones realizadas en este entregable deben quedar resueltas y documentadas.
2. **Dashboard predictivo en Dash Plotly, desplegado en Render,** con tres secciones: **Contexto**, **EDA** y **Modelos**. Debe contener, por modelo, la mejor combinación de metodologías de optimización y balances (si corresponde) entrenadas en el Entregable 2.
3. **Modelo original:** un modelo que no haya sido usado en la literatura para el problema de aplicación seleccionado, que no provenga de un simple `import`, que se implemente algorítmicamente y que se compare estadísticamente (mínimo tres pruebas de hipótesis) contra los modelos benchmark, mostrando que es superior.
4. **Informe técnico en Jupyter Book** con el detalle completo de la implementación.

Se evaluarán el rigor metodológico, la calidad de ingeniería, la honestidad científica en la afirmación de novedad y la calidad visual y comunicativa del producto final.

2. Panorama General del Entregable

La Figura 1 resume la ruta de trabajo. Los bloques A y B dependen del Entregable 2; el bloque C (modelo original) es el componente de investigación; el bloque D valida estadísticamente al bloque C; el bloque E integra todo en el dashboard; el bloque F lo hace público; y el bloque G documenta todo el proceso.

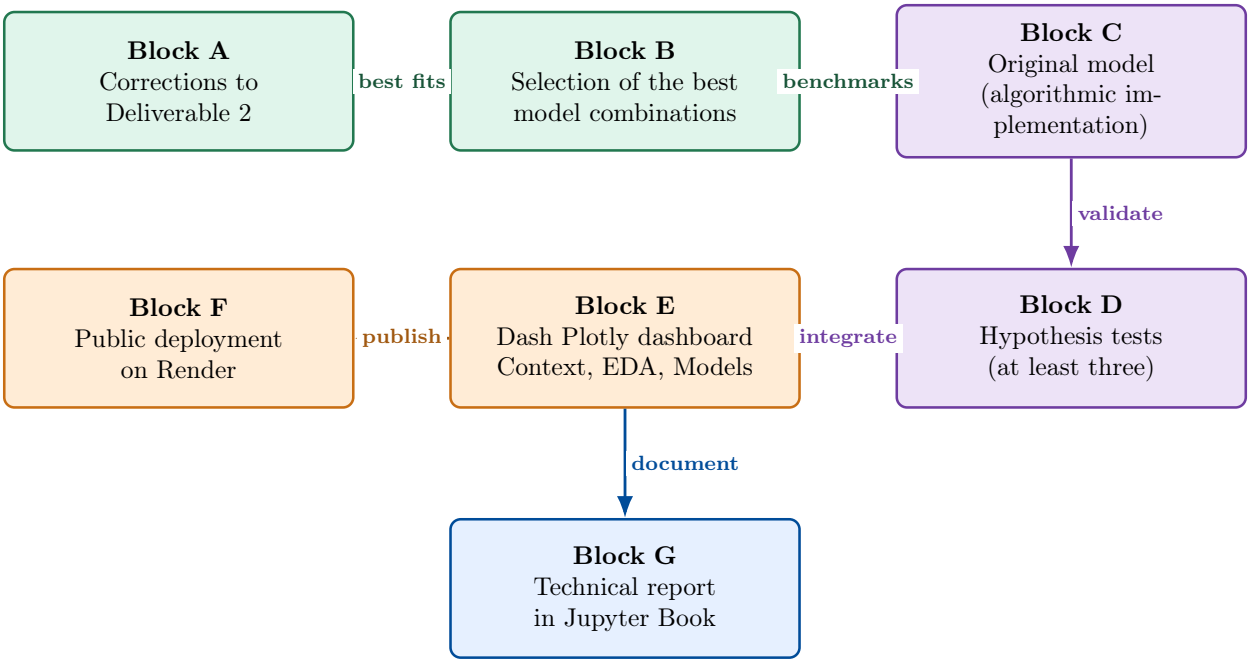


Figura 1: Flujo general del Entregable 3. Los bloques A y B reutilizan los resultados del Entregable 2; el bloque C aporta el modelo original; el bloque D lo valida; los bloques E y F lo ponen en producción y el bloque G documenta todo el proceso de forma reproducible.

3. Bloque A: Correcciones del Entregable 2

El estudiante debe resolver **todas** las observaciones realizadas en la retroalimentación del segundo entregable. Esto incluye, sin limitarse a ello, las correcciones al ETL, al EDA, a la validación anidada, a la implementación de las técnicas de optimización, a la comparación estadística y a la interpretabilidad.

- **Tabla de trazabilidad de correcciones (obligatoria):** una tabla en el Jupyter Book con las columnas *observación del docente*, *acción realizada*, *ubicación en el Jupyter Book* y *evidencia (figura, tabla o fragmento de código)*. Toda observación sin fila en esta tabla se considera no corregida.

- **Resultados actualizados:** si una corrección modifica algún resultado (por ejemplo, la eliminación de una fuga de datos, que suele reducir la métrica), todas las tablas, figuras y pruebas estadísticas que dependan de ese resultado deben recalcularse. No se aceptan correcciones parciales que dejen resultados inconsistentes en el informe.
- **Honestidad ante los cambios:** si el ranking de modelos cambia tras la corrección, debe reportarse y discutirse. Los cambios desfavorables no deben ocultarse.

4. Bloque B: Selección de las Mejores Combinaciones

Del universo de 140 combinaciones del Entregable 2 (Sección 2 de dicho entregable), el dashboard no debe incluir todas, sino las **mejores combinaciones**, definidas con criterios estadísticos y no por apreciación visual.

4.1. Criterio de selección

- Para **cada uno de los 7 modelos base** de la tarea seleccionada, se elige la mejor combinación (técnica de balanceo, solo en clasificación, y método de optimización) según la **métrica media del bucle externo** de la validación anidada.
- Se identifica además la **mejor combinación global** y se verifica su pertenencia al conjunto de finalistas de la comparación estadística del Entregable 2 (Friedman $\rightarrow$ Nemenyi en clasificación; MCS en regresión).
- Se documenta en una tabla maestra el motivo de cada selección, con la métrica como **media $\pm$ desviación estándar**.

4.2. Reentrenamiento final y serialización

- Una vez fijados los hiperparámetros óptimos, el pipeline completo (preprocesamiento + balanceo + modelo) se **reentrena sobre el conjunto de entrenamiento completo** y se serializa con `joblib` (o `skops`) en la carpeta `artifacts/`. El dashboard **nunca** debe reentrenar modelos ni ejecutar optimización de hiperparámetros: solo carga artefactos ya entrenados.
- Guardar junto a cada modelo un archivo `metadata.json` con: versión de las librerías, semilla, hiperparámetros finales, esquema de columnas de entrada (nombres, tipos, rangos válidos) y métricas del bucle externo.

- Verificar que el modelo serializado produce, sobre un conjunto de prueba fijo, **exactamente las mismas predicciones** que el modelo en memoria (prueba de ida y vuelta, o *round-trip test*).
- Cuidar el tamaño de los artefactos: el plan gratuito de Render tiene memoria limitada y los repositorios de GitHub tienen un límite por archivo. Si un modelo es demasiado pesado (por ejemplo, un Random Forest con miles de árboles profundos), se debe justificar y aplicar compresión (`joblib.dump(..., compress=3)`), reducción de árboles equivalente en desempeño o Git LFS.

5. Bloque C: El Modelo Original

Este es el componente central del entregable y el que define el tope de la calificación (Sección 12). Un modelo se considera **original** únicamente si cumple simultáneamente las tres condiciones siguientes.

Condiciones del modelo original

R1. Novedad en la aplicación: el modelo no ha sido usado en la literatura científica para resolver el **problema de aplicación seleccionado** (mismo fenómeno y misma tarea de predicción).

R2. Implementación algorítmica propia: el modelo no puede obtenerse mediante una línea del tipo `from libreria import Modelo`. Debe existir un proceso de implementación algorítmica programado por el estudiante (clase propia, reglas de actualización, función de pérdida, regla de combinación, etc.).

R3. Superioridad estadísticamente verificada: el modelo debe obtener un mejor score de clasificación o un menor error de regresión que los modelos benchmark, y la diferencia debe ser estadísticamente significativa según **al menos tres pruebas de hipótesis** adecuadas al problema (Sección 6).

5.1. Qué cuenta y qué no cuenta como modelo original

Cuadro 1: Criterios de admisibilidad para valorar la originalidad de la propuesta (modelos no neuronales)

Se acepta como contribución original	**No se acepta como contribución original**
Clase propia, compatible con `scikit-learn` (`fit`, `predict`, `predict_proba`), que programa desde cero una regla de aprendizaje o de combinación nueva, con formulación matemática explícita, derivación de la regla de actualización y justificación teórica de cada decisión de diseño.	Importar un modelo de una librería (`lightgbm`, `catboost`, `ngboost`, `tabpfn`, etc.) y llamarlo con otros hiperparámetros.
Contribución metodológica transferible: el mecanismo nuevo se formula de manera general, se enuncia como algoritmo con entradas, salidas y criterio de parada, y es aplicable a otros problemas de la misma tarea.	Adaptación ad hoc cuyo único mérito es que **nadie la ha aplicado a este problema**. La novedad solo de aplicación, sin mecanismo algorítmico nuevo, no es suficiente.
Sustento teórico verificable de al menos una propiedad: convergencia o terminación, consistencia, cota de error o de generalización, propiedades de la función objetivo, o complejidad temporal y espacial demostrada. Las demostraciones y los supuestos se incluyen en el informe.	Afirmaciones teóricas sin demostración, o justificaciones basadas solo en la intuición o en el buen desempeño empírico.
Ensamble con **meta-aprendiz o regla de ponderación diseñados y programados** por el estudiante (pesos según la región del espacio de entrada, la incertidumbre o el error local), con regla de actualización derivada y contrastada contra reglas estándar (promedio simple, pesos por error global, *stacking* lineal).	Usar `StackingClassifier`, `VotingRegressor`, `BaggingRegressor` o similares con los modelos del Entregable 2 sin una regla nueva de combinación.
Variante de boosting, bagging, bosques aleatorios o SVM con función de pérdida, criterio de partición, esquema de pesos o kernel propios, donde el **núcleo del algoritmo** (construcción del árbol, resolución de la optimización o regla de actualización) esté programado por el estudiante.	Cambiar la pérdida, el kernel o el criterio de partición mediante un parámetro de la librería, o reutilizar un `DecisionTreeRegressor` de `scikit-learn` como componente interno sin modificar su lógica.
Modelo híbrido no neuronal con **al menos un componente central** original y verificable, y un **estudio de ablación** que demuestre su aporte medible frente a las variantes sin él.	Encadenar modelos existentes en un `Pipeline` sin contribución algorítmica propia, o hibridar componentes cuyo aporte individual no se demuestra.

Continúa en la página siguiente

Se acepta como contribución original	No se acepta como contribución original
Diferenciación explícita frente al trabajo cercano: tabla comparativa con cinco a diez métodos afines (por familia algorítmica, no solo por aplicación) que precise qué es nuevo, qué se conserva y por qué los métodos existentes no resuelven la limitación abordada.	Declaración de novedad que solo reporta que no hubo resultados para la cadena de búsqueda, sin analizar métodos afines de otras aplicaciones.
Evaluación amplia y justa: además del problema principal, **al menos tres conjuntos de datos adicionales** de la misma tarea y de naturaleza distinta, mismas particiones para todos los modelos, **presupuesto de optimización equivalente** y comparación con métodos recientes del estado del arte además de los vistos en clase.	Evaluar en un solo conjunto de datos, dar más presupuesto de ajuste al modelo propio que a los benchmarks, o compararlo solo contra modelos débiles o mal ajustados.
Robustez, sensibilidad y limitaciones: sensibilidad a los hiperparámetros propios, estabilidad ante múltiples semillas, comportamiento bajo ruido, desbalance o menor tamaño de muestra, análisis de errores y discusión franca de dónde el modelo no supera a los benchmarks.	Reportar solo escenarios favorables, omitir fallos o presentar un valor puntual de la métrica sin su variabilidad.
Reproducibilidad completa: código público y versionado, semillas, entorno fijado, datos o instrucciones exactas de obtención y pruebas unitarias que verifican las propiedades del algoritmo.	Resultados que no se reproducen a partir del repositorio y del libro entregados.
Modelos probabilísticos o basados en kernels, vecindarios, árboles, reglas, optimización convexa o metaheurísticas, implementados con `numpy` y `scipy` como únicas herramientas numéricas.	**Cualquier red neuronal o componente neuronal** (MLP, CNN, RNN, LSTM, Transformer, autoencoder, *TabNet*, atención, *neural ODE*, etc.), aunque se programe a mano, y el uso de `torch`, `tensorflow`, `keras`, `jax` o `MLPClassifier`/`MLPRegressor`.

Sobre la afirmación de novedad: la novedad no se declara, se **demuestra** mediante una revisión sistemática de literatura documentada (Sección 5.2). Afirmar novedad sin evidencia de búsqueda, o afirmar originalidad en un modelo cuya implementación proviene de una librería o de código copiado de un repositorio, se considera una falta de integridad académica y se califica conforme a la Sección 13.

5.2. Protocolo de revisión de literatura (evidencia de novedad)

La revisión debe quedar documentada en un capítulo del Jupyter Book y cumplir lo siguiente:

- Consultar **al menos tres bases de datos** (por ejemplo, Scopus o Web of Science, Google Scholar, y arXiv, SSRN o IEEE Xplore, según el área del problema).
- Usar **al menos cinco cadenas de búsqueda** que combinen (i) el nombre o la familia del modelo propuesto, (ii) el problema de aplicación y (iii) sinónimos en inglés y en español. Registrar la fecha de la consulta.
- Completar una **matriz de novedad** como la mostrada en la Tabla 2, revisando título, resumen y, cuando corresponda, texto completo de los resultados más cercanos.
- Redactar una **declaración de novedad** precisa: "al mejor de nuestro conocimiento y según la búsqueda descrita, no se encontraron estudios que apliquen el modelo propuesto a [problema]". Mencionar explícitamente los trabajos más cercanos y en qué se diferencia la propuesta de cada uno.

Cuadro 2: Plantilla de matriz de novedad (ejemplo con datos ficticios)

Base de datos	**Cadena de búsqueda**	**Result.**	**Revis.**	**Relev.**	**Observación**
Scopus	`"modelo X" AND "problema Y"`	12	12	1	Estudio cercano usa otra arquitectura
Google Scholar	`modelo X problema Y forecasting`	340	40	0	Sin aplicación directa
arXiv	`model X problem Y`	7	7	0	Solo trabajos teóricos

5.3. Ciclo de desarrollo del modelo original

La Figura 2 muestra la secuencia que debe seguirse. El modelo original **debe pasar por exactamente el mismo protocolo de validación anidada, las mismas particiones externas y la misma semilla** que los modelos benchmark; de lo contrario, la comparación estadística no es válida.

5.4. Requisitos de la implementación

- **Clase propia** que herede de `BaseEstimator` y de `ClassifierMixin` o `RegressorMixin`, con los métodos `fit`, `predict` y, en clasificación, `predict_proba`. Esto garantiza que

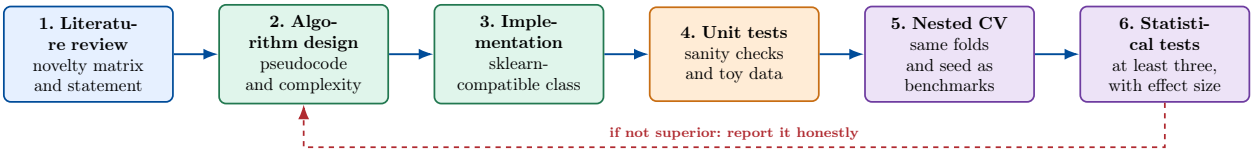


Figura 2: Ciclo de desarrollo del modelo original. La flecha punteada indica que, si el modelo no resulta superior, el estudiante puede iterar sobre el diseño del algoritmo, pero debe reportar de forma honesta el resultado final, aunque sea desfavorable.

el modelo se integre con `Pipeline`, `cross_val_score` y la validación anidada del Entregable 2.

- **Pseudocódigo formal** del algoritmo en el Jupyter Book (entradas, salidas, pasos, criterio de parada), con la notación matemática de cada paso.
- **Análisis de complejidad** asintótica en notación $O(\cdot)$ para entrenamiento e inferencia, contrastada con el tiempo empírico al variar n y p, en línea con la Sección de Optimización Computacional del Entregable 2.
- **Pruebas unitarias** (`pytest`) que verifiquen como mínimo: (i) que el modelo aprende en un conjunto sintético donde la solución es conocida, (ii) la reproducibilidad con la misma semilla, (iii) el manejo correcto de entradas inválidas y (iv) la compatibilidad con `sklearn.base.clone`.
- **Hiperparámetros del modelo propio** optimizados con al menos uno de los métodos del Entregable 2 (Bayesiano o Genético recomendados), dentro de la validación anidada.
- **Estudio de ablación:** cuando el modelo tenga varios componentes, mostrar el aporte de cada uno retirándolo por separado.
- **Interpretabilidad:** aplicar SHAP (o una alternativa justificada) al modelo original y compararlo con el mejor benchmark.
- **Prohibido** copiar implementaciones de repositorios o de modelos generados por herramientas de IA sin comprensión demostrable. Durante la sustentación, el estudiante deberá explicar y modificar en vivo cualquier parte del algoritmo.

Esqueleto sugerido de la clase (el estudiante debe sustituir la lógica marcada por su algoritmo propio):

```
import numpy as np                                    # libreria para operaciones numericas con
    arreglos
from sklearn.base import BaseEstimator, ClassifierMixin   # clases base que dan compatibilidad con scikit-
    learn
```

```python
from sklearn.utils.validation import check_X_y, check_array, check_is_fitted  # funciones de validacion de datos

class ModeloOriginal(BaseEstimator, ClassifierMixin):     # define la clase propia heredando las clases base
    def __init__(self, n_etapas=50, tasa=0.1, random_state=42):  # constructor: solo guarda hiperparametros
        self.n_etapas = n_etapas                          # numero de iteraciones del algoritmo
        self.tasa = tasa                                  # factor de aprendizaje o de actualizacion
        self.random_state = random_state                  # semilla para que los resultados sean reproducibles

    def fit(self, X, y):                                  # metodo de entrenamiento
        X, y = check_X_y(X, y)                            # valida y convierte los datos a arreglos numpy
        self.classes_ = np.unique(y)                      # guarda las clases observadas (exigido por sklearn)
        rng = np.random.default_rng(self.random_state)    # generador aleatorio con semilla fija
        self.estado_ = []                                 # lista donde se guardaran los componentes aprendidos
        for t in range(self.n_etapas):                    # ciclo principal del algoritmo propio
            raise NotImplementedError("Implementar aqui la regla de aprendizaje propia")  # logica del estudiante
        return self                                       # convencion de sklearn: fit devuelve el propio objeto

    def predict_proba(self, X):                           # metodo que devuelve probabilidades por clase
        check_is_fitted(self, "estado_")                  # verifica que el modelo ya fue entrenado
        X = check_array(X)                                # valida los datos de entrada
        raise NotImplementedError("Implementar aqui la regla de combinacion o de salida")  # logica del estudiante

    def predict(self, X):                                 # metodo que devuelve la clase predicha
        proba = self.predict_proba(X)                     # calcula las probabilidades de cada clase
        return self.classes_[np.argmax(proba, axis=1)]    # elige la clase con mayor probabilidad
```

5.5. Validez del modelo original: árbol de decisión

La Figura 3 sintetiza cómo se determina el tope de calificación del proyecto a partir de las tres condiciones R1, R2 y R3.

6. Bloque D: Verificación Estadística de la Superioridad

Para que el modelo original sea considerado **superior** (condición R3), no basta una métrica puntual mejor: la ventaja debe ser estadísticamente significativa y no trivial. Esta sección define el protocolo obligatorio.

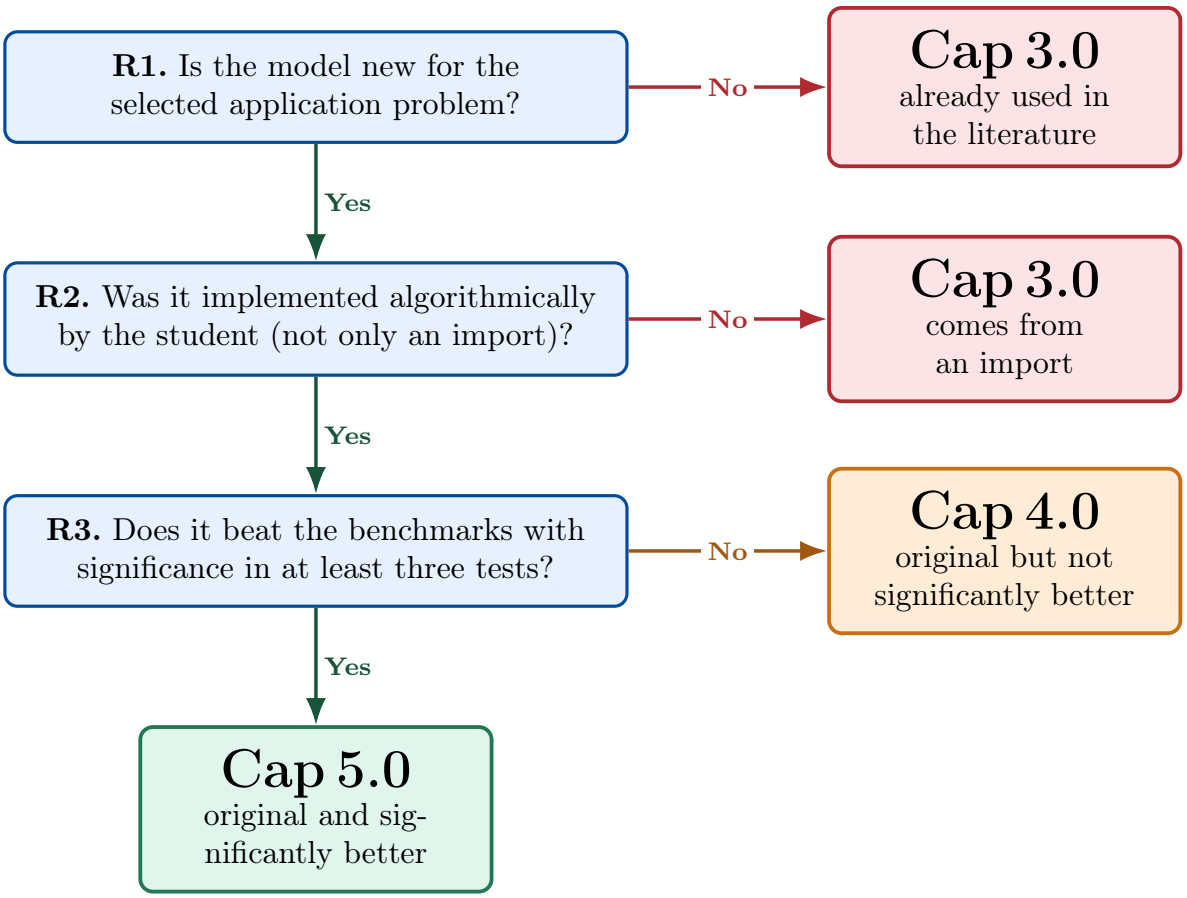


Figura 3: Árbol de decisión para el tope de calificación según las condiciones R1 (novedad), R2 (implementación propia) y R3 (superioridad estadísticamente significativa).

6.1. Definición operativa de superioridad

Se considera que el modelo original **supera** a los benchmarks cuando se cumplen **simultáneamente**:

1. La métrica media del bucle externo es mejor (mayor score en clasificación, menor RMSE o MAE en regresión) que la de la mejor combinación benchmark y que la de la mejor combinación de cada uno de los 7 modelos base incluidos en el dashboard.

2. **Al menos tres pruebas de hipótesis distintas** (de las admisibles en la Tabla 3) rechazan la hipótesis nula de igual desempeño, con p-valor **ajustado por comparaciones múltiples** (Holm-Bonferroni o Benjamini-Hochberg) menor que $\alpha = 0.05$ (menor que 0.10 para el MCS, por convención del procedimiento).

3. El **tamaño del efecto** no es despreciable: $|\delta_{\text{Cliff}}| \geq 0.147$ (umbrales de Romano et al.: < 0.147 despreciable, < 0.33 pequeño, < 0.474 mediano, en adelante grande) o $|d_{\text{Cohen}}| \geq 0.2$ cuando aplique.

6.2. Reglas metodológicas para que las pruebas sean válidas

- **Pareado estricto:** el modelo original y los benchmarks deben evaluarse sobre **las mismas particiones externas**, con la misma semilla global y el mismo preprocesamiento.
- **Potencia estadística:** con solo 5 o 10 pliegues, las pruebas no paramétricas tienen poca potencia. Se recomienda usar **validación cruzada externa repetida** (por ejemplo, 5 repeticiones de 5 pliegues) y, para las pruebas basadas en remuestreo, la corrección de varianza de Nadeau y Bengio, dado que las estimaciones de pliegues no son independientes.
- **Verificar los supuestos** de cada prueba (normalidad, estacionariedad de la serie de diferencias de pérdida, independencia, anidamiento de modelos) y justificar por qué la prueba elegida es la adecuada.
- **Adecuar la prueba al tipo de datos:** las pruebas de Diebold-Mariano, Giacomini-White, Clark-West, MCS y SPA suponen un orden temporal de los errores. Si el problema es de corte transversal, deben sustituirse por las pruebas de la columna correspondiente de la Tabla 3.
- **Control de comparaciones múltiples:** reportar siempre el p-valor crudo y el ajustado, y el número total de comparaciones realizadas.
- **Intervalos de confianza** de la diferencia de desempeño mediante bootstrap (BCa para métricas asimétricas; bootstrap estacionario en datos temporales).

6.3. Reporte obligatorio de resultados

- Una **tabla resumen** con el modelo original y los benchmarks: media $\pm$ desviación estándar del bucle externo, intervalo de confianza bootstrap y tiempo de entrenamiento e inferencia.
- Una **tabla de pruebas de hipótesis** con las columnas: prueba, hipótesis nula, supuestos verificados, estadístico, p-valor crudo, p-valor ajustado, tamaño del efecto y conclusión.

Cuadro 3: Pruebas admisibles para el requisito R3. Se deben aplicar **al menos tres** por problema, de las cuales al menos una debe ser un procedimiento global o con control de *data snooping*

Tarea	**Datos independientes (corte transversal)**	**Datos con dependencia temporal**
Clasificación	Friedman con Nemenyi o Wilcoxon-Holm y diagrama de diferencia crítica; DeLong para AUC; McNemar sobre el conjunto de prueba; prueba t corregida de Nadeau-Bengio sobre validación repetida; prueba bayesiana de rangos con signo (región de equivalencia práctica, ROPE).	Los mismos procedimientos aplicados con particiones que respeten el orden temporal (*walk-forward*); McNemar y DeLong solo si se justifica la validez del supuesto de independencia.
Regresión	Wilcoxon de rangos con signo sobre las diferencias de pérdida por pliegue; prueba de permutación pareada sobre errores absolutos o cuadráticos; prueba t corregida de Nadeau-Bengio; bootstrap pareado BCa de la diferencia de RMSE.	Model Confidence Set (MCS); Giacomini-White o Clark-West (según haya anidamiento); Diebold-Mariano con corrección de Harvey-Leybourne-Newbold; Reality Check o SPA; bootstrap estacionario.

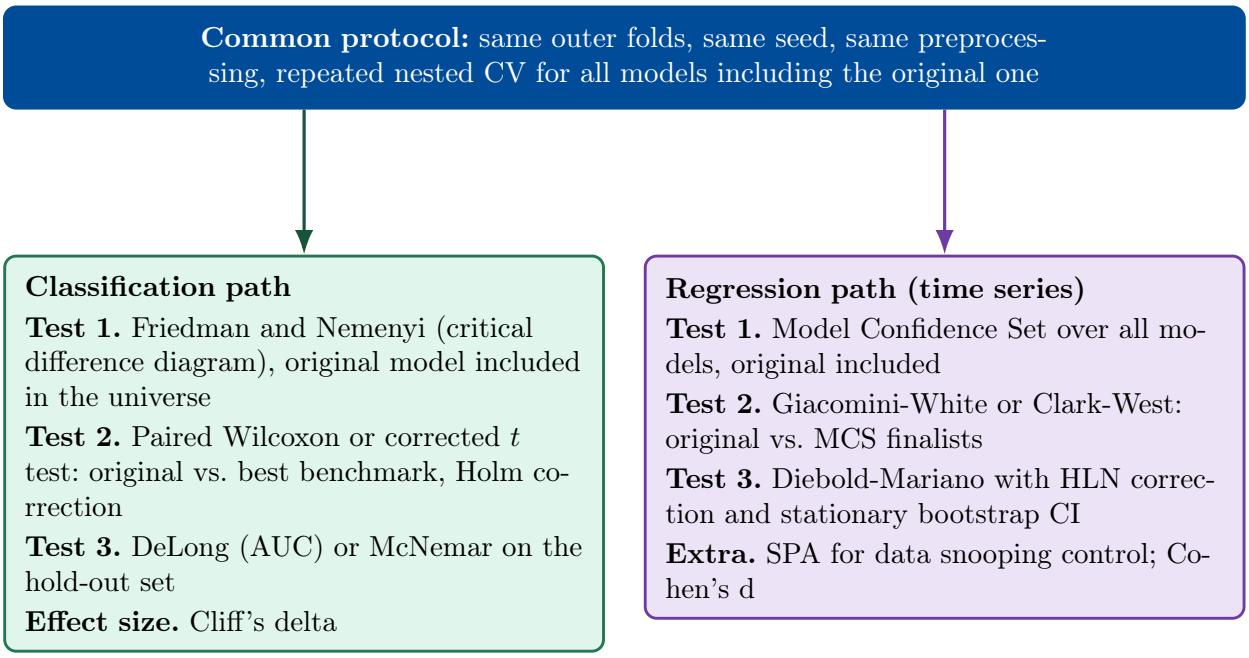


Figura 4: Ruta sugerida de pruebas de hipótesis para el modelo original. Los tres bloques de cada columna cubren la exigencia mínima de tres pruebas; los resultados deben reportarse con p-valor crudo, p-valor ajustado, intervalo de confianza y tamaño del efecto.

- El **diagrama de diferencia crítica** (clasificación) o la **tabla del MCS** (regresión) con el modelo original incluido.
- Una **discusión honesta**: si el modelo no resulta superior o la diferencia no es significativa, debe indicarse y analizarse la causa probable (tamaño de muestra, ruido, falta de capacidad del modelo, etc.). Esto no invalida el trabajo; define el tope de 4.0 conforme a la Sección 12.

7. Bloque E: Dashboard Predictivo en Dash Plotly

7.1. Objetivo

Construir una aplicación web interactiva, atractiva y robusta, con la calidad de comunicación esperable de un proyecto de Machine Learning, que permita a una persona no experta **comprender el problema, explorar los datos y usar los modelos para predecir**. El dashboard es un producto de ciencia de datos, no una colección de gráficos: cada elemento debe responder a una pregunta de un usuario real.

7.2. Arquitectura y estructura del repositorio

Se exige una aplicación **multipágina** (`use_pages=True`), con una página por sección, y una separación clara entre interfaz, lógica de datos y artefactos de modelo.

```
proyecto-ml-final/
|-- app.py                  # punto de entrada: crea la app y expone "server"
|-- pages/
|   |-- contexto.py         # seccion 1: Contexto
|   |-- eda.py              # seccion 2: EDA
|   `-- modelos.py          # seccion 3: Modelos
|-- src/
|   |-- original_model.py   # clase del modelo original
|   |-- data.py             # carga y cache de datos
|   `-- plots.py            # funciones que construyen las figuras
|-- artifacts/              # modelos serializados y metadata.json
|-- data/                   # datos procesados (livianos)
|-- assets/                 # CSS, logo, tipografias
|-- tests/                  # pruebas unitarias con pytest
|-- book/                   # fuentes del Jupyter Book
|-- requirements.txt        # versiones exactas
`-- render.yaml             # configuracion de despliegue
```

7.3. Contenido mínimo de cada sección

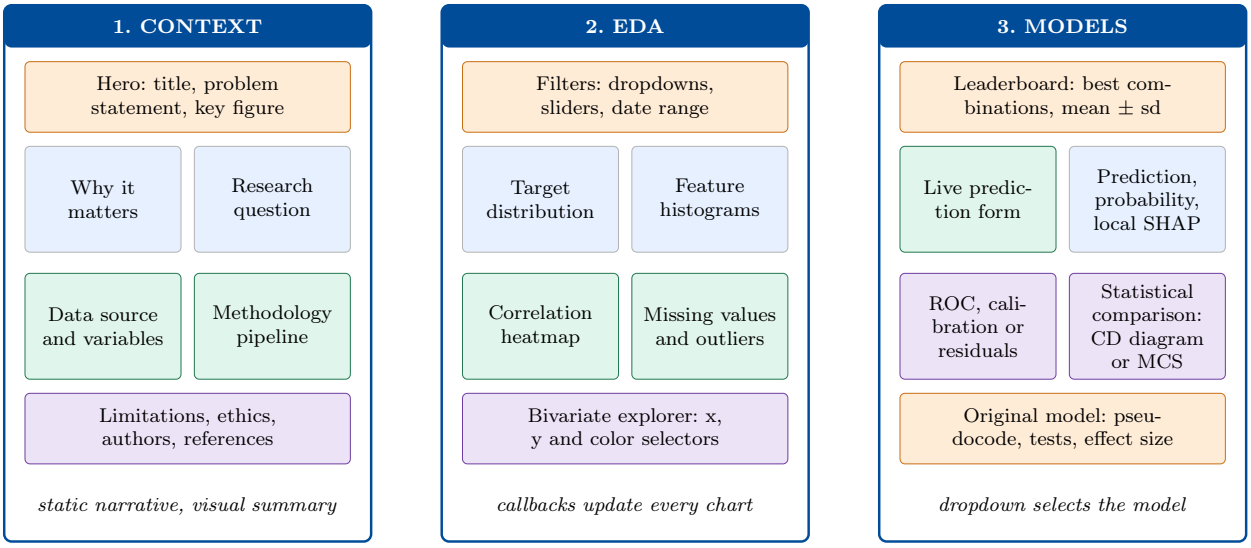


Figura 5: Maqueta (*wireframe*) orientativa de las tres secciones obligatorias del dashboard. El estudiante puede mejorar el diseño, pero debe conservar el contenido mínimo de cada sección.

7.3.1. Sección 1: Contexto

- **Planteamiento del problema** y su relevancia social, económica o científica, con al menos una cifra o hecho de contexto citado con su fuente.
- **Pregunta de investigación** y objetivos, formulados de manera clara.
- **Descripción de los datos:** fuente, periodo, número de observaciones y variables, diccionario de variables, variable objetivo y tipo de tarea (clasificación o regresión).
- **Resumen de la metodología:** un diagrama del pipeline (ETL, EDA, modelos, optimización, validación anidada, comparación estadística, modelo original).
- **Limitaciones, consideraciones éticas y de sesgo**, autores, versión y fecha de actualización.

7.3.2. Sección 2: EDA

- **Filtros globales interactivos** (`dcc.Dropdown`, `dcc.RangeSlider`, `dcc.DatePickerRange`) que actualicen todos los gráficos de la página mediante *callbacks*.
- Distribución de la variable objetivo (y balance de clases en clasificación).
- Distribuciones univariadas y relaciones bivariadas, con un **explorador** donde el usuario elija las variables de los ejes y el color.

- Matriz de correlación, valores faltantes y valores atípicos.
- En series de tiempo: descomposición, ACF/PACF y pruebas de estacionariedad.
- Cada gráfico debe acompañarse de **una frase de interpretación** (*insight*), no solo de la figura.

7.3.3. Sección 3: Modelos

- **Tabla de líderes** (`dash_table.DataTable`) con las mejores combinaciones de la Sección 4: modelo, balanceo, optimizador, métrica como media ± desviación estándar del bucle externo y tiempo de inferencia. Debe permitir ordenar y filtrar.
- **Predicción en vivo:** formulario de entrada generado a partir del esquema de variables, con validación de rangos y tipos, y un selector de modelo. La salida debe mostrar la predicción, la **probabilidad** (clasificación) o un **intervalo de predicción** (regresión), y una **explicación local SHAP** de esa predicción.
- **Diagnóstico del modelo seleccionado:** en clasificación, matriz de confusión, curvas ROC y de precisión-recall y diagrama de calibración; en regresión, valores reales frente a predichos, residuos, ACF de residuos y histograma.
- **Interpretabilidad global** (resumen SHAP) para el modelo seleccionado.
- **Comparación estadística:** diagrama de diferencia crítica (clasificación) o tabla del MCS (regresión) con el modelo original incluido.
- **Subsección del modelo original:** descripción, pseudocódigo, diferencias respecto a la literatura, curva de convergencia o de aprendizaje, tabla de pruebas de hipótesis con p-valores ajustados, tamaño del efecto y un veredicto explícito de superioridad o no superioridad.

7.4. Criterios de calidad de diseño y de ingeniería

- **Diseño visual coherente:** usar `dash-bootstrap-components` con un tema, una paleta limitada (máximo cinco colores), tipografía consistente, espaciado uniforme y una barra de navegación fija. Evitar saturación de colores y gráficos sin título o sin unidades.
- **Gráficos Plotly de calidad:** títulos, ejes con unidades, leyendas legibles, información en *hover* y plantilla (`template`) unificada. Todo texto de las figuras debe ser legible en pantallas pequeñas.

- **Responsividad:** el dashboard debe usarse correctamente en escritorio y en tablet; verificar con el sistema de rejilla (`dbc.Row`, `dbc.Col`).
- **Rendimiento:** cargar datos y modelos **una sola vez** al iniciar; usar `dcc.Store` o `functools.lru_cache` para cálculos repetidos; evitar callbacks que recalculen todo en cada interacción; usar `prevent_initial_call` y `dash.no_update` cuando corresponda.
- **Robustez:** validar entradas del usuario, manejar errores con mensajes claros (`dbc.Alert`) y evitar que una entrada inválida detenga la aplicación.
- **Accesibilidad:** contraste suficiente, no depender solo del color para transmitir información, y textos alternativos descriptivos.
- **Código limpio:** callbacks organizados por página, funciones de graficación reutilizables en `src/plots.py` y ausencia de datos, rutas o credenciales escritos directamente en el código.

Esqueleto sugerido de `app.py`:

```
import dash                                          # framework principal para construir la aplicacion
    web
import dash_bootstrap_components as dbc              # componentes visuales con estilo profesional
from dash import html                                # elementos HTML (titulos, parrafos, contenedores)

app = dash.Dash(                                     # crea la aplicacion
    __name__,                                        # nombre del modulo actual
    use_pages=True,                                  # activa la navegacion por paginas (carpeta pages/)
    external_stylesheets=[dbc.themes.FLATLY],        # aplica un tema visual de Bootstrap
    title="Proyecto ML Final",                       # titulo que se ve en la pestana del navegador
)                                                    # fin de la configuracion de la app

server = app.server                                  # expone el servidor WSGI que usara gunicorn en
    Render

navbar = dbc.NavbarSimple(                           # barra de navegacion superior
    children=[                                       # lista de enlaces de la barra
        dbc.NavItem(dbc.NavLink("Contexto", href="/")),          # enlace a la pagina de contexto
        dbc.NavItem(dbc.NavLink("EDA", href="/eda")),            # enlace a la pagina de analisis
    exploratorio
        dbc.NavItem(dbc.NavLink("Modelos", href="/modelos")),    # enlace a la pagina de modelos
    ],                                               # fin de la lista de enlaces
    brand="Proyecto ML Final",                       # nombre que aparece a la izquierda de la barra
    color="primary",                                 # color principal del tema
    dark=True,                                       # texto claro sobre fondo oscuro
)                                                    # fin de la barra de navegacion

app.layout = dbc.Container(                          # contenedor principal de toda la aplicacion
    [navbar, dash.page_container],                   # barra fija arriba y la pagina activa debajo
    fluid=True,                                      # usa todo el ancho disponible de la pantalla
)                                                    # fin del layout
```

```
if __name__ == "__main__":                    # solo se ejecuta al correr el archivo localmente
    app.run(debug=True)                       # arranca el servidor local en modo desarrollo
```

8. Bloque F: Despliegue en Render

El dashboard debe estar **publicado y accesible mediante una URL pública** en Render, sin necesidad de instalación local para quien lo evalúe.



Figura 6: Ruta de despliegue: del desarrollo local a la URL pública en Render. Cada *push* a la rama principal puede disparar un nuevo despliegue automático.

8.1. Pasos de despliegue

1. Verificar que la aplicación corre localmente con `python app.py` y que `app.server` está expuesto.
2. Crear `requirements.txt` con **versiones exactas** (incluyendo `gunicorn`) y fijar la versión de Python.
3. Subir el repositorio a GitHub (público o privado con acceso para Render).
4. En Render, crear un **Web Service** conectado al repositorio, con comando de construcción `pip install -r requirements.txt` y comando de inicio `gunicorn app:server`.
5. Verificar los *logs* de construcción, corregir errores de dependencias y comprobar que todas las páginas y callbacks funcionan en la URL pública.

Archivo de configuración sugerido (`render.yaml`):

```
services:
  - type: web                      # tipo de servicio: aplicacion web
    name: dash-ml-final            # nombre del servicio en Render
    runtime: python                # entorno de ejecucion
    buildCommand: pip install -r requirements.txt   # instala las dependencias
    startCommand: gunicorn app:server               # arranca el servidor de produccion
    envVars:
      - key: PYTHON_VERSION        # fija la version de Python
        value: 3.11.9
```

Consideraciones del plan gratuito: los servicios del plan gratuito de Render pueden suspenderse tras un periodo de inactividad, de modo que el primer acceso puede tardar en responder, y tienen límites de memoria y de CPU. Verificar los límites vigentes en la documentación oficial de Render. Buenas prácticas: abrir la URL varios minutos antes de la sustentación, mantener los artefactos livianos, no entrenar nada dentro del servidor y manejar con cuidado el consumo de memoria al cargar los modelos. Si el despliegue falla por límites de recursos, el estudiante debe reducir el tamaño de los artefactos y documentar la decisión.

Ejemplo de *callback* de predicción (se incluye para ilustrar el patrón; cada estudiante debe adaptarlo a su problema y variables):

```
import joblib                                          # libreria para cargar modelos guardados
import pandas as pd                                    # libreria para manejar tablas de datos
from dash import Input, Output, State, callback        # objetos necesarios para definir callbacks

MODELO = joblib.load("artifacts/best_model.joblib")    # carga el modelo una sola vez al iniciar el
    servidor

@callback(                                             # declara una funcion que reacciona a eventos
    Output("salida-pred", "children"),                 # componente donde se mostrara el resultado
    Input("boton-predecir", "n_clicks"),               # el callback se dispara al pulsar el boton
    State("entrada-x1", "value"),                      # lee el valor de la variable 1 sin disparar el
     callback
    State("entrada-x2", "value"),                      # lee el valor de la variable 2 sin disparar el
     callback
    prevent_initial_call=True,                         # evita ejecutar el callback al cargar la pagina
)                                                      # fin de la declaracion del callback
def predecir(n_clicks, x1, x2):                        # funcion que calcula la prediccion
    if x1 is None or x2 is None:                       # revisa si falta alguna entrada
        return "Complete todos los campos."            # mensaje de ayuda para el usuario
    fila = pd.DataFrame([{"x1": x1, "x2": x2}])        # arma una tabla de una fila con las entradas
    proba = MODELO.predict_proba(fila)[0, 1]           # calcula la probabilidad de la clase positiva
    return f"Probabilidad estimada: {proba:.1%}"       # devuelve el texto que vera el usuario
```

9. Bloque G: Informe Técnico en Jupyter Book

El informe debe documentar **toda la implementación** y ser reproducible: cualquier persona debe poder reconstruir los resultados ejecutando el libro. Se evalúa tanto el contenido científico como la calidad de la escritura y de la presentación.

9.1. Estructura mínima de capítulos

Cuadro 4: Estructura mínima del Jupyter Book

N.°	Capítulo	Contenido mínimo
0	Introducción	Resumen ejecutivo, problema, objetivos, mapa del libro y URL del dashboard desplegado.
1	Trazabilidad de correcciones	Tabla de observaciones del Entregable 2 y acciones realizadas (Sección 3).
2	Datos: ETL y EDA	ETL y EDA mejorados, con interpretación de cada figura.
3	Modelos benchmark	Definición, hiperparámetros y resultados de las mejores combinaciones (Sección 4).
4	Revisión de literatura y novedad	Matriz de búsqueda, declaración de novedad y trabajos cercanos (Sección 5.2).
5	Modelo original	Motivación, formulación matemática, pseudocódigo, complejidad, código comentado, pruebas unitarias y ablación.
6	Validación estadística	Protocolo, supuestos, tres o más pruebas, ajustes, intervalos y tamaños del efecto (Sección 6).
7	Dashboard y despliegue	Arquitectura, capturas de pantalla, decisiones de diseño, guía de despliegue y URL pública.
8	Conclusiones y limitaciones	Hallazgos, veredicto sobre R1, R2 y R3, limitaciones, trabajo futuro y declaración de uso de herramientas de IA.
9	Reproducibilidad y bibliografía	Entorno, semillas, tabla maestra de experimentos y referencias en formato BibTeX.

9.2. Requisitos técnicos del libro

- Archivos `_config.yml`, `_toc.yml` y `references.bib`, con citas y referencias cruzadas funcionales.
- Celdas de código **ejecutables**, debidamente comentadas, que reproduzcan tablas y figuras. Para cómputos muy costosos se aceptan resultados precalculados y versionados, siempre que el código que los genera esté incluido y documentado.
- Todas las figuras con título, ejes etiquetados, unidades y una interpretación en el texto. Todas las tablas con descripción y referencia en el cuerpo del texto.
- Ecuaciones numeradas y definidas con la sintaxis de MyST (`$$ ... $$`).
- El libro debe construirse **sin errores** con `jupyter-book build book/` y publicarse (por ejemplo, en GitHub Pages) o entregarse como carpeta HTML comprimida junto con el repositorio.

Ejemplo de `_toc.yml`:

```
format: jb-book                 # formato de libro de Jupyter Book
root: intro                     # archivo de entrada
chapters:
  - file: 01_correcciones       # trazabilidad de correcciones
  - file: 02_datos              # ETL y EDA
  - file: 03_benchmarks         # modelos benchmark
  - file: 04_literatura         # revision de literatura y novedad
  - file: 05_modelo_original    # modelo original
  - file: 06_validacion         # pruebas de hipotesis
  - file: 07_dashboard          # dashboard y despliegue
  - file: 08_conclusiones       # conclusiones y limitaciones
  - file: 09_reproducibilidad   # entorno, semillas y bibliografia
```

10. Lista de Entregables

1. **URL pública del dashboard** desplegado en Render, funcional el día de la sustentación.
2. **Repositorio de GitHub** con la estructura de la Sección 7, `README.md` (descripción, instalación, uso y despliegue), `requirements.txt` con versiones exactas y semilla global documentada.
3. **Jupyter Book** (fuentes y versión compilada o publicada) con los capítulos de la Sección 9.
4. **Código del modelo original** (`src/original_model.py`) con *docstrings* y pruebas unitarias.
5. **Tabla maestra de experimentos** (CSV o parquet) con las 140 combinaciones del Entregable 2, el modelo original y las métricas del bucle externo, tiempos y semillas.
6. **Artefactos de los modelos** (`artifacts/`) con `metadata.json`.
7. **Declaración de autoría y uso de herramientas** (incluidas herramientas de IA), firmada por el equipo, en el último capítulo del libro.

11. Sustentación

La sustentación tendrá una duración de **15 minutos** de exposición, seguidos de una sesión de preguntas, y se realizará sobre el dashboard **desplegado** (demostración en vivo). Debe cubrir:

- Problema, datos y hallazgos principales del EDA (2 minutos).
- Mejores combinaciones de modelos y demostración de una predicción en vivo con su explicación local (3 minutos).
- Modelo original: idea, algoritmo, sustento teórico y evidencia de novedad (4 minutos).
- Validación estadística: pruebas, p-valores ajustados, tamaño del efecto, robustez y veredicto (4 minutos).
- Conclusiones, limitaciones y trabajo futuro (2 minutos).

Durante las preguntas, el docente podrá solicitar la explicación o modificación en vivo de cualquier parte del algoritmo del modelo original y de las pruebas estadísticas. La imposibilidad de explicar el propio código es evidencia en contra de las condiciones R1 y R2.

12. Evaluación y Calificación

La nota del entregable está en la escala de **0 a 5**. Se calcula en dos pasos: primero se obtiene una nota ponderada según la rúbrica y luego se aplica el **tope** definido por el estado del modelo original.

12.1. Tope de calificación según el modelo original

Cuadro 5: Tope de calificación según las condiciones R1, R2 y R3

Tope	**Condición**	**R1, R2, R3**
5.0	El modelo nunca se ha usado en el problema de aplicación, no proviene de un `import` y supera de forma estadísticamente significativa (mínimo tres pruebas) en error o score a los modelos benchmark (SOTA) estudiados en clase.	R1 sí, R2 sí, R3 sí
4.0	El modelo nunca se ha usado en el problema de aplicación y no proviene de un `import`, pero **no** supera de forma estadísticamente significativa en error o score a los modelos benchmark.	R1 sí, R2 sí, R3 no
3.0	El modelo ya se ha usado para la aplicación, o su implementación proviene de un `import` sin programación algorítmica propia, y su precisión no es superior a la de los benchmark.	R1 no o R2 no

Regla para casos intermedios: el tope lo determina la **condición incumplida de menor nivel**. Si el modelo ya se ha usado en la literatura para el problema (R1 no) o proviene de un `import` (R2 no), el tope es **3.0**, incluso si por casualidad su desempeño resulta superior; la superioridad estadística solo eleva el tope por encima de 4.0 cuando R1 y R2 se cumplen. Si el modelo cumple R1 y R2 pero no R3, el tope es 4.0.

12.2. Rúbrica ponderada

Cuadro 6: Rúbrica de evaluación (ponderación sobre 100 %)

Componente	**Peso**	**Criterios principales**
Correcciones del Entregable 2	10 %	Tabla de trazabilidad completa; resultados recalculados y consistentes; ausencia de fuga de datos.
Mejores combinaciones y artefactos	10 %	Criterio de selección estadístico; reentrenamiento final; serialización verificada con prueba de ida y vuelta.
Dashboard: Contexto y EDA	13 %	Contenido mínimo; interactividad con callbacks; interpretación de cada figura.
Dashboard: Modelos	10 %	Tabla de líderes, predicción en vivo con explicación local, diagnósticos y comparación estadística.
Dashboard: diseño, UX y despliegue	7 %	Calidad visual, responsividad, rendimiento, robustez y URL pública funcional.
Modelo original: revisión y diseño	7 %	Matriz de novedad, declaración de novedad, formulación y pseudocódigo.
Modelo original: implementación	8 %	Clase propia compatible con `scikit-learn`, complejidad, pruebas unitarias, ablación y comprensión demostrada.
Pruebas de hipótesis	15 %	Mínimo tres pruebas adecuadas, supuestos verificados, ajuste por comparaciones múltiples, intervalos y tamaño del efecto.
Informe en Jupyter Book	15 %	Estructura completa, reproducibilidad, calidad de la redacción y de las figuras, bibliografía.
Sustentación	5 %	Claridad, dominio técnico, demostración en vivo y respuestas a preguntas.
Total	**100 %**	

Cálculo de la nota final:

$$\text{Nota final} \;=\; \text{mín}\Big(\text{Tope},\; 5.0 \times \tfrac{\text{Puntaje ponderado}}{100}\Big)$$

donde el **Tope** se determina con la Tabla anterior (5.0, 4.0 o 3.0). Un entregable perfecto con modelo original no superior en forma significativa tiene entonces una nota máxima de 4.0.

13. Integridad Académica

- Toda afirmación de novedad debe estar respaldada por la revisión documentada de la Sección 5.2. Una afirmación de novedad falsa o sin evidencia se trata como falta de integridad académica y el modelo se evalúa como si no cumpliera R1.
- Se permite consultar documentación, artículos y repositorios como **inspiración**, citándolos. Copiar código sin atribución o presentar como propia una implementación ajena se sanciona conforme al reglamento estudiantil.
- Si se usan herramientas de IA, deben declararse en el último capítulo del libro, indicando para qué se usaron. El estudiante es responsable de comprender y poder defender cada línea entregada.
- Reportar un resultado desfavorable de forma honesta **no** se penaliza más allá del tope definido; manipular particiones, semillas o pruebas para obtener significancia sí constituye una falta grave.

14. Cronograma Sugerido

15. Conclusiones

El estudiante debe demostrar:

- Capacidad de corregir y mejorar su propio trabajo a partir de la retroalimentación.
- Capacidad de llevar modelos de Machine Learning a un producto interactivo, desplegado y comunicable.
- Capacidad de investigación: diseñar, implementar algorítmicamente y validar un modelo propio frente al estado del arte estudiado.

Cuadro 7: Cronograma orientativo de trabajo (las fechas de entrega serán confirmadas por el docente)

Semana	Actividad	Producto parcial
1	Correcciones del Entregable 2; selección de mejores combinaciones.	Tabla de trazabilidad; artefactos serializados.
2	Revisión de literatura; diseño del modelo original.	Matriz de novedad; pseudocódigo.
3	Implementación del modelo original y pruebas unitarias.	Clase propia funcional.
4	Validación anidada y pruebas de hipótesis.	Tablas de resultados y veredicto R3.
5	Desarrollo del dashboard y despliegue en Render.	URL pública funcional.
6	Jupyter Book, revisión final y ensayo de sustentación.	Libro compilado y entrega final.

- Rigor estadístico para sustentar, o refutar con honestidad, la superioridad de su propuesta.
- Calidad de documentación y reproducibilidad propias de un entorno científico y profesional.

Este entregable enfatiza que, en entornos reales y de investigación, un modelo no está terminado cuando se entrena: está terminado cuando es **reproducible, validado estadísticamente, interpretable y puesto a disposición de quien lo necesita**.

Referencias Sugeridas

Referencias

[1] Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.

[2] Benavoli, A., Corani, G., Demšar, J., y Zaffalon, M. (2017). Time for a change: a tutorial for comparing multiple classifiers through Bayesian analysis. *Journal of Machine Learning Research*, 18(77), 1–36.

[3] DeLong, E. R., DeLong, D. M., y Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves. *Biometrics*, 44(3), 837–845.

[4] Nadeau, C., y Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52, 239–281.

[5] Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.

[6] Diebold, F. X., y Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business and Economic Statistics*, 13(3), 253–263.

[7] Harvey, D., Leybourne, S., y Newbold, P. (1997). Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2), 281–291.

[8] Giacomini, R., y White, H. (2006). Tests of conditional predictive ability. *Econometrica*, 74(6), 1545–1578.

[9] Clark, T. E., y West, K. D. (2007). Approximately normal tests for equal predictive accuracy in nested models. *Journal of Econometrics*, 138(1), 291–311.

[10] Hansen, P. R., Lunde, A., y Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2), 453–497.

[11] Hansen, P. R. (2005). A test for superior predictive ability. *Journal of Business and Economic Statistics*, 23(4), 365–380.

[12] Politis, D. N., y Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303–1313.

[13] Romano, J., Kromrey, J. D., Coraggio, J., y Skowronek, J. (2006). Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? *Annual Meeting of the Florida Association of Institutional Research*.

[14] Lundberg, S. M., y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[15] Akiba, T., Sano, S., Yanase, T., Ohta, T., y Koyama, M. (2019). Optuna: a next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD*, 2623–2631.